CODSOFT DATA SCIENCE INTERNSHIP

TASK 2 – MOVIE RATING PREDICTION WITH PYTHON

IMDb Movies India Dataset • Regression Project

1. Objective

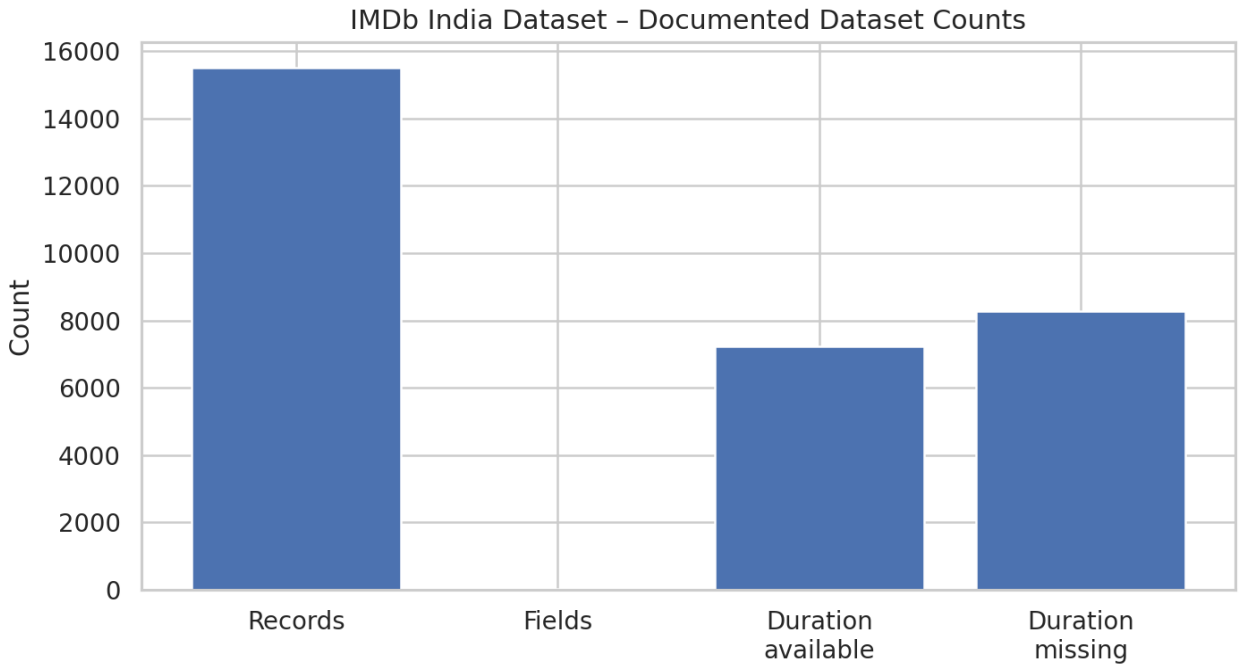
Build a regression model that predicts a movie rating using information such as year, duration, genre, director, actors and votes. The CodSoft brief describes this as a regression problem.

2. Dataset

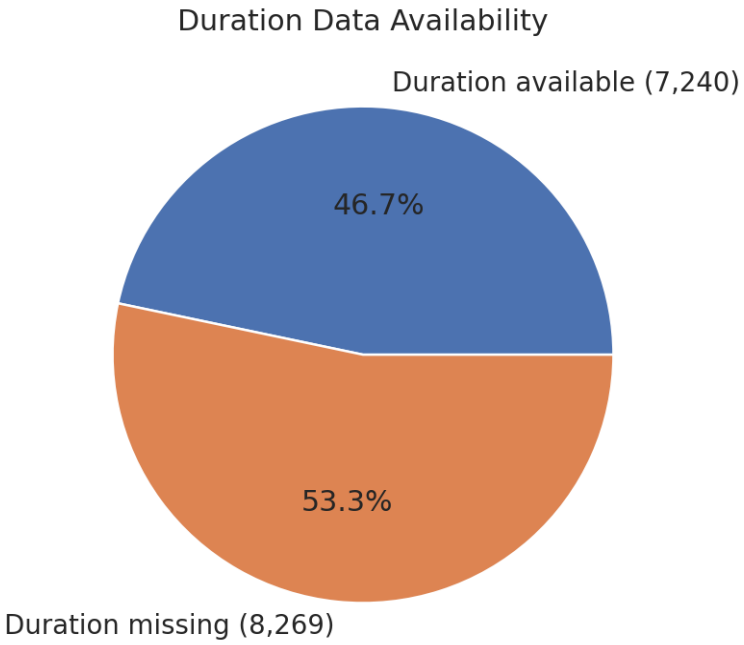
The exact IMDb Movies India.csv file is publicly listed in a GitHub repository used for CodSoft Task 2. The source page reports 15,510 lines including the header and a file size of about 1.32 MB. A project using this dataset documents the fields Name, Year, Duration, Genre, Rating, Votes, Director, Actor 1, Actor 2 and Actor 3.

Item	Documented value
Records	15,509
Fields	10
Duration values available after removing missing entries	7,240
Target	Rating

3. Data Visualizations

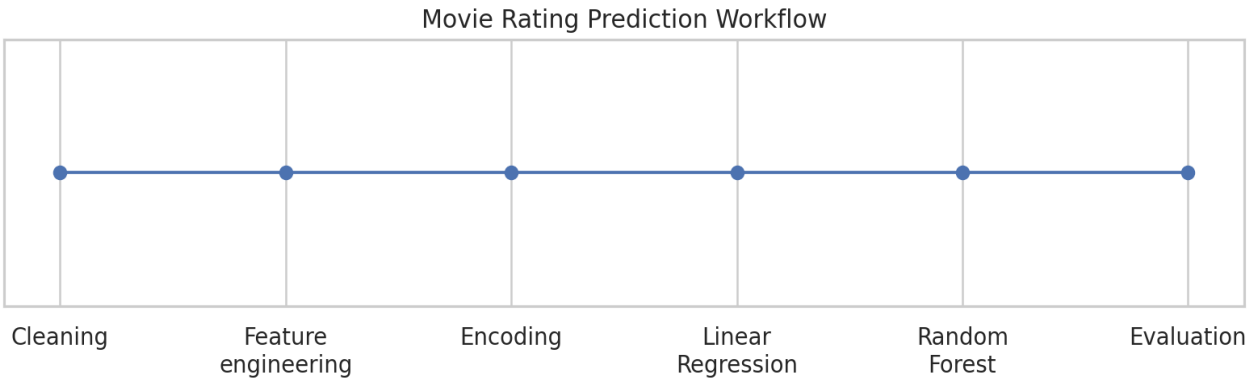


Dataset counts documented by the public dataset/project sources.



The public analysis reports 7,240 movies remaining after removing records with missing Duration.

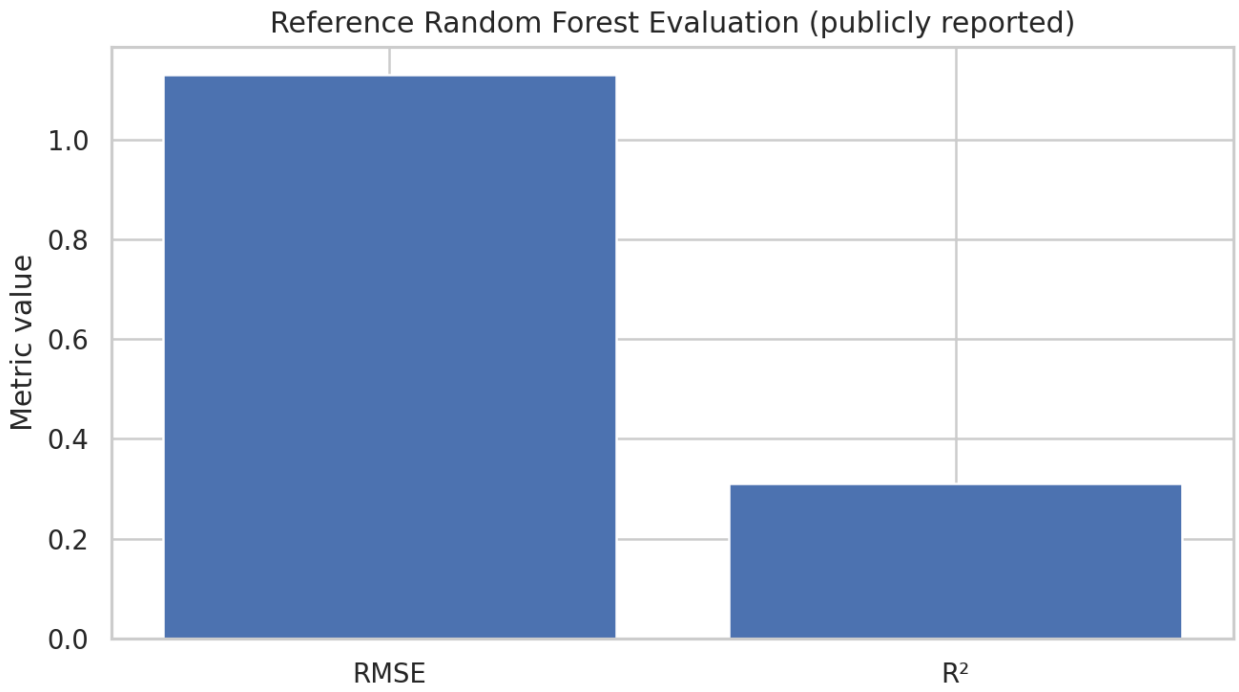
4. Machine Learning Approach



Preprocessing: numeric conversion for Year, Duration and Votes; missing-value handling; one-hot encoding for categorical columns; train/test split; regression modeling.

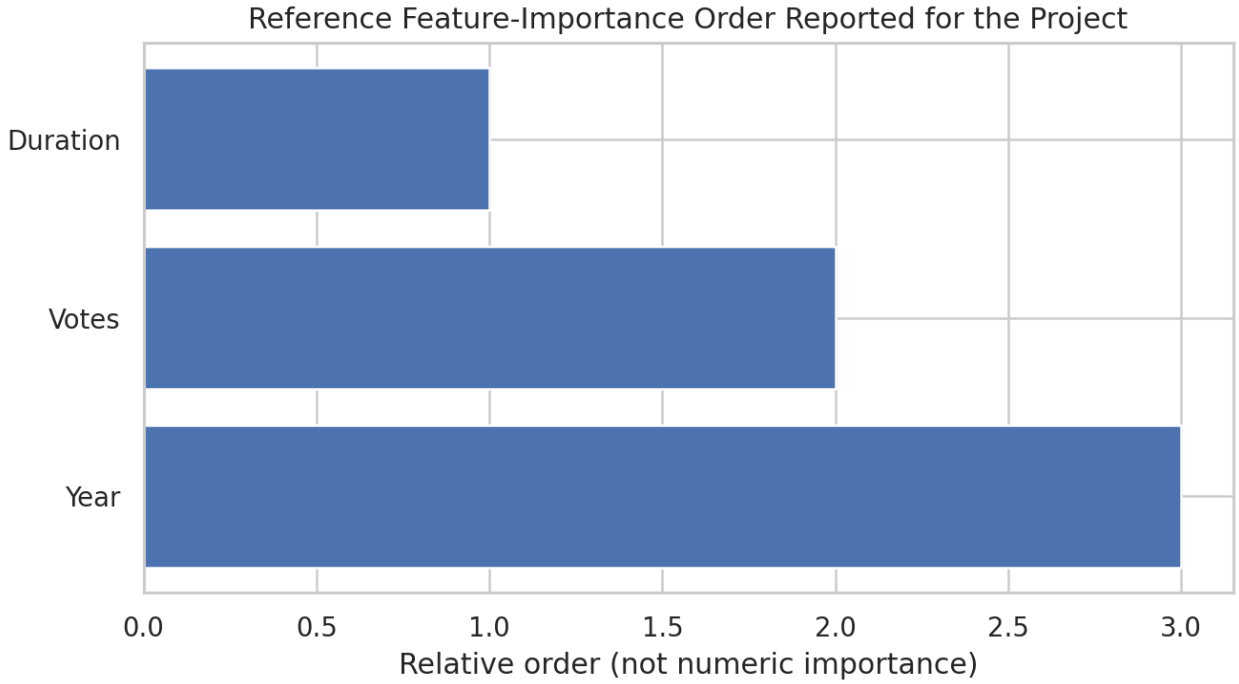
5. Models

• Linear Regression – baseline regression model.
• Random Forest Regressor – nonlinear ensemble model used as the final/reference model in the public CodSoft project.
The public project description also mentions Gradient Boosting and Random Forest tuning.



Reference result reported publicly for a CodSoft implementation using this dataset: Random Forest $R^2 = 0.31$ and RMSE = 1.13. These values are included as a reference report, not claimed as a fresh local execution in this environment.

6. Important Features



A public project using the dataset reports Year, Votes and Duration as the most influential features. The graphic shows their reported order only; it is not a fabricated numeric feature-importance chart.

7. Complete Python Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("IMDb Movies India.csv")
df.columns = df.columns.str.strip()

# Convert numeric columns
df["Year"] = pd.to_numeric(
df["Year"].astype(str).str.extract(r"(\d{4})")[0],
errors="coerce"
)
df["Duration"] = pd.to_numeric(
df["Duration"].astype(str).str.extract(r"(\d+)")[0],
errors="coerce"
)
df["Votes"] = pd.to_numeric(
df["Votes"].astype(str).str.replace(",", "", regex=False),
errors="coerce"
)
df["Rating"] = pd.to_numeric(df["Rating"], errors="coerce")

# Keep rows with a known target
df = df.dropna(subset=["Rating"])

features = [
"Year", "Duration", "Votes",
"Genre", "Director", "Actor 1", "Actor 2", "Actor 3"
]
X = df[features]
y = df["Rating"]

numeric = ["Year", "Duration", "Votes"]
categorical = ["Genre", "Director", "Actor 1", "Actor 2", "Actor 3"]

preprocess = ColumnTransformer([
("num", SimpleImputer(strategy="median"), numeric),
("cat", Pipeline([
("imputer", SimpleImputer(strategy="most_frequent")),
("onehot", OneHotEncoder(handle_unknown="ignore"))
]), categorical)
])

X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.20, random_state=42
)

# Linear Regression
linear = Pipeline([
("prep", preprocess),
("model", LinearRegression())
])
linear.fit(X_train, y_train)
linear_pred = linear.predict(X_test)

print("Linear MAE:",
mean_absolute_error(y_test, linear_pred))
print("Linear RMSE:",
np.sqrt(mean_squared_error(y_test, linear_pred)))
print("Linear R2:",
r2_score(y_test, linear_pred))

# Random Forest
rf = Pipeline([
("prep", preprocess),
("model", RandomForestRegressor(
n_estimators=200,
random_state=42,
n_jobs=-1
))
])
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

print("RF MAE:",
mean_absolute_error(y_test, rf_pred))
print("RF RMSE:",
np.sqrt(mean_squared_error(y_test, rf_pred)))
print("RF R2:",
r2_score(y_test, rf_pred))
```

```
# Visualizations
sns.histplot(df["Rating"].dropna(), kde=True)
plt.title("Movie Rating Distribution")
plt.show()

df["Year"].value_counts().sort_index().plot(kind="line")
plt.title("Movies Released by Year")
plt.xlabel("Year")
plt.ylabel("Number of Movies")
plt.show()

genres = (df["Genre"].dropna()
.str.split(",").explode()
.str.strip().value_counts().head(10))
sns.barplot(x=genres.values, y=genres.index)
plt.title("Top 10 Movie Genres")
plt.show()

sns.scatterplot(
data=df.dropna(subset=["Duration","Rating"]),
x="Duration", y="Rating"
)
plt.title("Duration vs Rating")
plt.show()

sns.scatterplot(
data=df.dropna(subset=["Votes","Rating"]),
x="Votes", y="Rating"
)
plt.title("Votes vs Rating")
plt.show()

sns.heatmap(
df[["Year","Duration","Rating","Votes"]].corr(),
annot=True, fmt=".2f"
)
plt.title("Correlation Heatmap")
plt.show()

# Actual vs predicted
plt.scatter(y_test, rf_pred)
plt.xlabel("Actual Rating")
plt.ylabel("Predicted Rating")
plt.title("Actual vs Predicted Ratings")
plt.show()

# Residuals
residuals = y_test - rf_pred
plt.scatter(rf_pred, residuals)
plt.axhline(0, linestyle="--")
plt.xlabel("Predicted Rating")
plt.ylabel("Residual")
plt.title("Residual Plot")
plt.show()
```

8. Conclusion

The project covers dataset understanding, preprocessing, exploratory visualization, regression modeling and evaluation. The public source confirms the exact dataset structure, while the public CodSoft implementation documents the Random Forest reference result and feature-importance order. For a strict fresh execution, run the supplied code in Anaconda/Jupyter with the exact CSV file.

Sources: GitHub copy of IMDb Movies India.csv; public CodSoft Task 2 project documentation; public analysis of the same dataset. ■cite sources are not embedded in the PDF; URLs are listed in the accompanying chat.